上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/10/10

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

软件运行时

模块 5 · 重新认识软件（2/4）

饮 水 思 源 • 爱 国 荣 校

SJTU

本讲位置：从「实例」到「进程」

上一讲 · L-05

软件的生命周期：制品轴与实例轴

实例会被频繁替换，所以要停得优雅、起得快、不带状态

留下的线头：启动时间为什么是硬参数

本讲问题

实例轴上「服务中」的那个东西，在机器上到底是什么？它占了什么？上限由哪个资源给？

钩子

L-04 留下的题：选课高峰时，最先崩的是 Tomcat 线程池，还是数据库行锁？

第三节回答，课程实践用数字验证

本讲结构

1 一个服务在机器上是什么

2 语言运行时

3 并发模型与吞吐

插曲：延迟怎么说才对 · 思考题 2 发布

4 运行时的层级

5 课程实践：单机 Tomcat 的运行时画像

本模块 重新认识软件（L-05 ~ L-08）：制品（L-05）· 进程（本讲）· 资源的消费者（L-07）· 可度量的数（L-08）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、一个服务在机器上是什么

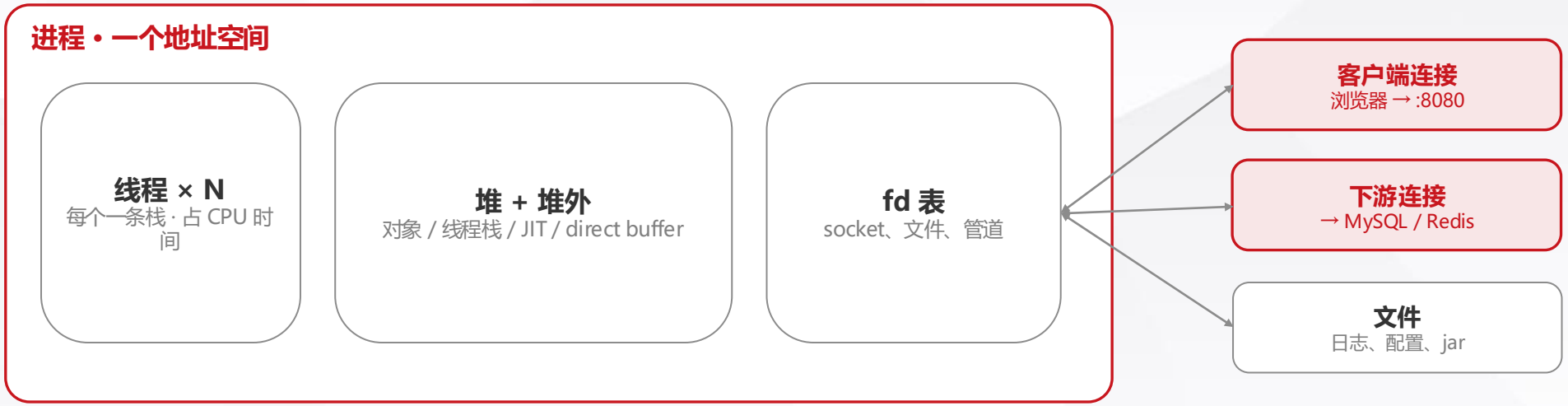
第六讲 · 软件运行时

饮水思源 · 爱国荣校

SJTU

操作系统视角 · 进程

从「实例」到「进程」



线程
占 CPU 时间和栈内存

内存
占物理页：堆、堆外、页缓存

fd
占连接与文件

要点 本节按这三样讲，每一样都能在 /proc/<pid>/ 下看到；L-07 讲每样的上限

来源：proc(5) man page；B. Gregg, Systems Performance, 2nd ed. (2020), Ch. 3

CPU：线程与调度

线程是调度单位

内核按时间片轮流把线程放上核

可运行线程多于核数就排队

利用率（核在忙）≠ 饱和（有线程在等核）

上下文切换

直接代价微秒级，间接代价是缓存被冲掉

线程多到一定程度，切换本身开始吃 CPU

怎么读数

一个进程的 %CPU 可以超过 100%（4 核最多 400%）

user 是你的代码，sys 是内核替你做的

load average 数的是可运行 + 不可中断的线程

问题	原因	对策
CPU 利用率 60%，响应却越来越慢	看的是平均；某些核或某些时段已饱和，或线程在等锁 / I/O	看运行队列和 load average；看线程状态；看 sys / user 比例
线程开到 2000，吞吐反而下降	上下文切换与缓存失效吃掉了 CPU	线程数按核数与阻塞比例定，不按并发用户数定（第三节）

要点 利用率是「核在忙」，饱和是「有人在等核」；慢的时候先分清在等 CPU 还是在等别的

来源：B. Gregg, Systems Performance, Ch. 6 CPUs；Linux sched(7)

操作系统视角 · 内存

内存：一个进程占了什么

进程的 RSS（真占的物理内存）

堆	堆外	共享库	页缓存
运行时管理的对象	线程栈 · 元数据 · JIT 代码 · direct buffer	libc、JVM 本体	内核用空闲内存缓存文件，可回收

虚拟内存是「申请的」，RSS 才是「真占的」

不算在进程里，但服务要靠它

问题	原因	对策
free 显示内存快没了，服务却没事	空闲内存被内核拿去做页缓存，需要时会让出	看 available，不看 free
堆设了 2 GB，进程却占 3 GB，容器里被杀	堆只是一部分：线程栈、元数据、JIT、direct buffer 都在堆外	容器限制 = 堆 + 堆外 + 余量；JDK 10+ 读 cgroup 限制自动调堆
进程突然消失，退出码 137	超过 cgroup 内存限制被 OOM killer 杀（137 = 128 + SIGKILL）	看 dmesg / OOMKilled；限制与实际占用对账

要点 「这个服务要多少内存」的答案不是 -Xmx，而是 RSS 的上限加上页缓存的需要

来源：B. Gregg, Systems Performance, Ch. 7 Memory；Linux proc(5)（/proc/meminfo、/proc/<pid>/status）

操作系统视角 • fd

连接与文件描述符

连接数 ≠ 请求数

每个 TCP 连接、每个打开的文件都是一个 fd；进程有上限（ulimit -n，常见默认 1024）

keep-alive 的空闲连接不发请求也占 fd 和缓冲区：2000 人 × 每人几条 = 上万个 fd

连接的两头

客户端连接（浏览器 → Tomcat）和下游连接（Tomcat → MySQL / Redis）各占一份

短连接频繁建拆会堆出大量 TIME_WAIT，临时端口也会用完

问题	原因	对策
高峰时报 Too many open files，CPU 却很闲	fd 上限 1024 被空闲 keep-alive 连接占满	提高进程与系统 fd 上限；缩短 keep-alive 超时；前面放反向代理收敛连接
新连接超时，老连接正常	服务端 maxConnections 到顶，新连接排在 accept 队列里	看 ss -s / ss -ltn 的 backlog；调 maxConnections 与 backlog；仍不够就横向扩

回指 思考题 1 里 Nginx 没说出口的收益：它替后端扛住上万条客户端连接，后端只看到少量复用的上游连接

来源：Linux ss(8)、ulimit；Apache Tomcat 10.1 HTTP Connector 文档（maxConnections、acceptCount）

怎么看：一页速查

想看什么	命令	看哪个数
进程占了多少 CPU / 内存	top；ps -o pid,nlwp,rss,%cpu	%CPU（可超 100%）、RSS、线程数 nlwp
CPU 是忙还是在排队	uptime；vmstat 1	load average；r 列（可运行）、b 列（阻塞）
内存到底还剩多少	free -m	available，不是 free
开了多少 fd、多少连接	ls /proc/<pid>/fd | wc -l；ss -tanp	fd 数；ESTAB / TIME-WAIT 计数
线程在干什么	top -H；jstack；pprof	哪些线程占 CPU；多少在等锁、等 I/O

要点 这些就是 2025 实验 2 的工具；L-07 讲每个数的上限，实验里自己量

来源：各命令 man page；2025 实验 2 指导书

操作系统视角 · 演示

现场演示：看一个真实的进程（3 分钟）

做什么

被观察对象：一个空载的 Tomcat（或 2025 实验 3 的 Go 后端）

① **top -H -p <pid>**

线程数、每个线程的 %CPU、状态 S / R

② **grep -E "VmRSS|Threads" /proc/<pid>/status**

RSS 与线程数——和 -Xmx 对一下

③ **free -m**

free 与 available 差多少

④ **ls /proc/<pid>/fd | wc -l；ss -s**

空载时就有几十个 fd；连接的状态分布

然后压 10 秒再看一遍

wrk -t4 -c200 -d10s http://<host>:8080/

预期看到

线程数不变（池是预分配的），但 R 状态的线程变多

RSS 略涨，远小于 200 × 1 MB——线程栈是按需提交的

ESTAB 连接 ≈ 200，压完以后 TIME-WAIT 一片

把这四个数记下来，课程实践里要用它们算上限

讲法 只看数字、不讲工具用法；每个数对应第 5 ~ 8 页的一样东西

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、语言运行时

第六讲 · 软件运行时

饮水思源 • 爱国荣校

SJTU

JVM：堆、GC、JIT、线程

堆与 GC

对象都在堆里，分代回收

GC 要停顿（STW），长短取决于回收器与堆大小

「堆越大越好」不成立

JIT

先解释执行，热点再编译

启动慢、预热后峰值高

L-05「启动时间是硬参数」的根源之一

平台线程

1:1 对应 OS 线程，各有一条栈（预留 1 MB，按需提交）

200 个不贵，2 万个就贵了

虚拟线程（JDK 21）

M:N 映射到少量 OS 线程，阻塞时让出

可以开百万个：阻塞写法、非阻塞成本

第三节说它改变了什么、没改变什么

问题	原因	对策
容器里 JVM 莫名被杀	-Xmx 之外还有元空间、代码缓存、线程栈、direct buffer	容器限制 ≥ 堆 + 堆外 + 余量；用 JDK 10+ 的容器感知
刚发布的实例前几分钟特别慢	JIT 还没预热	就绪检查里加预热（L-05）；或用 GraalVM 原生镜像换掉 JIT

要点 JVM 的资源画像：内存基线高、启动要预热、线程用 OS 线程——每一条都有代价，也都有新办法

来源：JEP 444 Virtual Threads；JDK 容器支持（JDK-8146115）；HotSpot GC Tuning Guide

Go · Node · Python：各自的并发单位

Go

goroutine：M:N 调度到 OS 线程，初始栈 KB 级，开十万个很平常

并发 GC，停顿短；静态二进制，启动毫秒级；多核自动用满

坑：goroutine 泄漏；GC 仍占 CPU

2025 实验 2、3 的后端就是 Go

Node.js

单线程事件循环 + libuv 线程池做 I/O；并发靠不阻塞

一段 CPU 密集代码会卡住所有请求 → worker threads 或多进程

多核要开多个进程

Python

GIL：同一时刻只有一个线程执行字节码（3.13 起有实验性的无 GIL 构建）

I/O 并发用 asyncio，CPU 并行靠多进程

典型部署 = 多个 worker 进程 × 少量线程

要点 并发单位不同 → 「一个实例能撑多少并发」的算法不同；下一页放在一张表里

来源：Go runtime scheduler 设计文档；Node.js docs, The Node.js Event Loop；PEP 703

同一个服务，不同的资源画像

运行时	并发单位	内存基线	启动时间	多核	峰值吞吐
JVM（JIT）	平台线程 / 虚拟线程	高（百 MB 起）	秒级 + 预热	自动	高
GraalVM 原生镜像	同上	低	毫秒级	自动	略低（无 JIT）
Go	goroutine	低（十 MB 级）	毫秒级	自动	高
Node.js	事件循环	中	亚秒	多进程	I/O 高，CPU 低
Python	进程 × 线程	中	亚秒	多进程	低

选运行时 = 选资源画像：同一个选课服务，JVM 写一台机器放 2 个实例，Go 写能放 10 个——这就是 L-07 与模块 6 要算的账

回收 L-03：Quarkus / GraalVM 的启动与内存，换来的是峰值吞吐与构建时间；表中为量级示意

判断 这项技术解决了什么问题，又增加了什么成本？

来源：L-03 讲义（Quarkus 对比）；GraalVM Native Image 文档

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

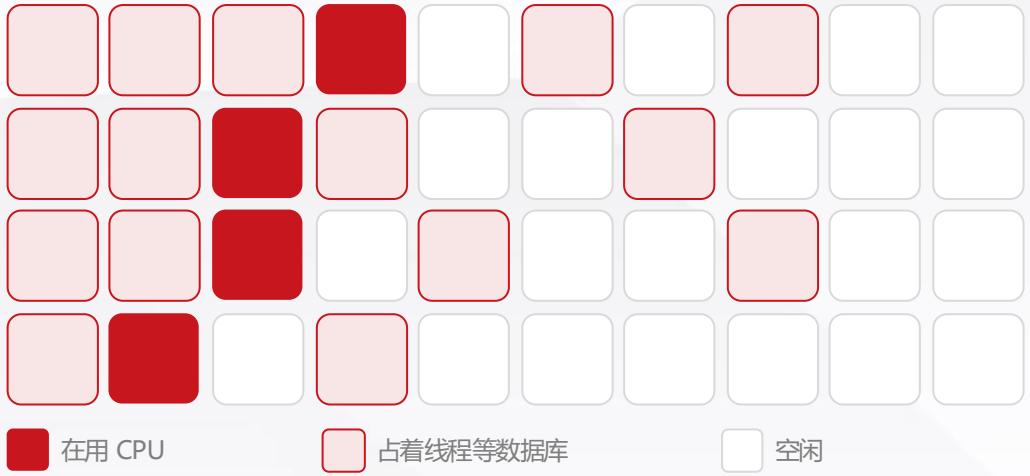
三、并发模型与吞吐

第六讲 · 软件运行时

饮 水 思 源 • 爱 国 荣 校

SJTU

一个请求占一个线程



maxThreads = 200：一个请求进来占一格，等数据库返回时这一格空转——不用 CPU，但不释放

Little’ s Law

在途请求 L = λ × W

一请求一线程 → 并发度上限 = 线程数 → λ_max = 线程数 / W

算两笔

200 线程、W = 50 ms → 4000 rps

数据库慢到 W = 500 ms → 只剩 400 rps

线程池先满，再排队，再超时

要点 线程池的上限不是常数，它随每个请求的等待时间变化——等待越久，线程池「越小」

来源：J. D. C. Little (1961)；Apache Tomcat 10.1 HTTP Connector（maxThreads 默认 200）

排队无处不在

accept 队列
内核 backlog / acceptCount 100

→

线程池
maxThreads 200，满了排队

→

连接池
等一个空闲的数据库连接

→

锁等待
热点行，一次只能过一个

每一级都是缓冲，每一级都加延迟；有界队列 + 超时 = 背压；无界队列 = 延迟无限涨、内存无限涨

问题	原因	对策
高峰过去很久，服务仍然慢	队列里积压的请求还在被处理，而客户端早已放弃	有界队列；请求带截止时间，过期直接丢
加大线程池，超时反而更多	排队的地方从线程池挪到了数据库	线程数按下游容量定；真正的瓶颈在下游

回收思考题 1 第二幕：单机 p50 2.1 s 时在途 5000 个请求，几乎全在排队——排队时间是重载下响应时间的全部来源

要点 队列是必要的缓冲，也是延迟的来源；没有上限的队列迟早把机器拖垮

来源：B. Gregg, Systems Performance, Ch. 2（排队论）；Tomcat acceptCount 默认 100

非阻塞与事件循环

epoll / NIO

少量线程管大量连接；等 I/O 时不占线程

并发上限从「线程数」变成「内存与 fd」

但 CPU 密集仍要核

事件循环里算 50 ms 的东西，所有连接一起等 50 ms

Node 的典型事故

Tomcat 与虚拟线程

Tomcat NIO 连接器接连接就是这样（acceptor + poller），业务线程仍是一请求一线程

虚拟线程让阻塞写法得到非阻塞成本，把「线程数」这一层上限基本抹掉

要点 非阻塞改变的是线程这一层的上限；CPU、连接池、锁的上限一个都没变——课程实践第 4 题

来源：Linux epoll(7)；Tomcat NIO connector 文档；JEP 444

连接池：更稀缺的资源

数据库那头也是线程

MySQL 一连接一线程，max_connections 默认 151

所以池不是越大越好：HikariCP 建议核数 × 2 量级，默认 10

池吞吐上限 = 池大小 / 每次占用时长

10 个连接、每次 8 ms → 1250 QPS，不管前面有多少线程

池满的表现是延迟（等连接），不是错误

问题	原因	对策
池从 10 调到 100，吞吐没变、数据库 CPU 飙了	数据库那头的并行度有限，多出来的连接在数据库里排队	池按数据库能力定，不按应用线程数定
偶发慢请求，数据库本身不慢	在等连接	看池的 wait 指标；减少每次占用时长（事务短一点）或适度加池

要点 监控里要看「等待获取连接」的时间；它经常是第一个到顶的上限

来源：HikariCP wiki, About Pool Sizing；MySQL 8.4 Reference, max_connections

并发模型 • 钩子

回答 L-04 的问题：线程池，还是行锁?

热点行行锁
提交被串行化
→ **300 TPS**
单机写吞吐到顶
→ **线程池 200**
提交在这里排队
→ **accept 队列 100**
再排一层
→ **超时 • 重试**
剩下的在客户端

← 原因在这头　　症状在这头 →

读路径（查课表、查余量）

每请求等待短，线程池远远不满

先到顶的是 CPU 或连接池——课程实践算

写路径（抢课提交）

热门课余量行的行锁把提交串行化：单机约 300 TPS，加机器的总上限也只有约 1200 TPS（思考题 1 第五幕，$s \approx 0.25$）

开放瞬间 2000 个提交涌来，在单机 300 TPS 的口子前排队——排队的地方就是线程池和 accept 队列

加线程只是把候车室修大

要点 症状在线程池，原因在行锁；真正的解在数据层（Redis 原子扣减、排队削峰、限流）——L-04 讨论第 2 题与思考题 1 第五幕在这里会合

来源：思考题 1 教师要点（第五幕数据）；L-03 Amdahl 定律

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

插曲·延迟怎么说才对

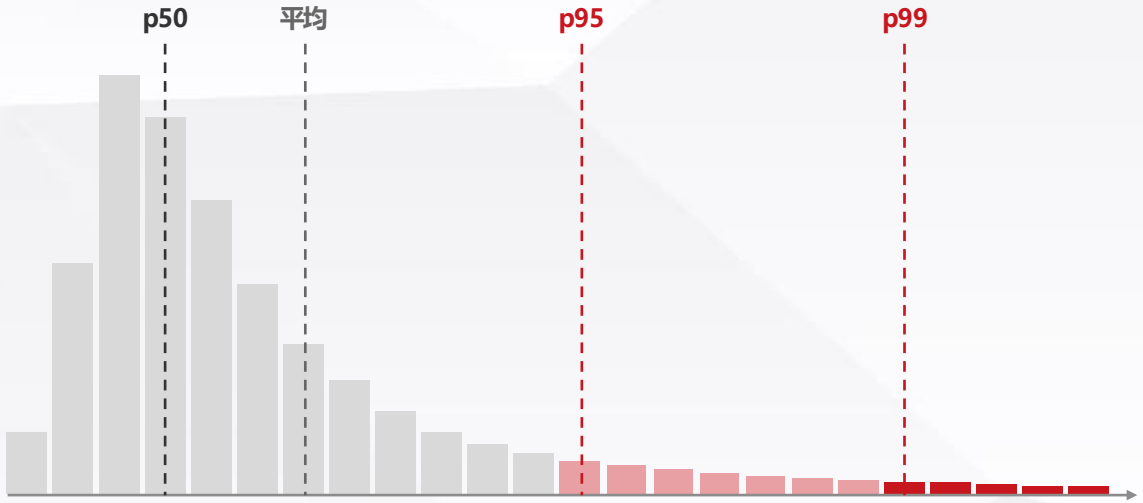
思考题 2 发布

饮水思源·爱国荣校

SJTU

插曲 · 百分位数

平均值骗人：p50 / p95 / p99



大部分请求挤在左边，一条长尾拖到右边；平均值被尾巴拉偏，p50 比平均值还小

定义

p99 = 把 100 个请求按快慢排好，第 99 个的耗时——「100 个里最慢的那个」

为什么关心那 1%

一次页面要几十个请求、一次会话几百个请求，碰到尾巴的概率远不止 1%

重度用户碰得最多——他们恰恰是最在乎的用户

说延迟要带百分位；SLO 写的是 p99 不是平均（L-08）

要点 平均值描述不了延迟；p50 看典型，p99 看尾巴

来源：J. Dean & L. Barroso, The Tail at Scale, CACM 56(2), 2013

扇出放大：1 – 0.99^N

一个页面并行调 N 个服务，每个的 p99 是 100 ms

页面至少碰到一个「慢于 100 ms」调用的概率 = 1 – 0.99^N

N	1 – 0.99^N	含义
1	1%	就是那个服务的 p99
12	11.4%	单个服务的 p99 成了页面的 p89
100	63.4%	大多数页面都会碰到慢调用

前提

各服务的慢事件相互独立

不独立时（共享同一个数据库）概率低一些，但一慢全慢

对策只点名，题里讨论

关键路径 · 超时与降级 · 对冲请求 · 重试预算

这就是思考题 2 第四题的全部数学；其余几题用 L-03、L-04 的内容即可作答

要点 扇出越大，尾巴越重；管住尾巴是分布式系统的日常

来源：Dean & Barroso (2013)；思考题 2 题面

思考题 2 发布：「14 个服务，160 毫秒」

场景

选课平台三年后拆成 14 个服务，首页 p50 反而从 180 ms 降到 160 ms

实习生的估算 14 × 20 = 280 ms，错在哪?

五幕五题

估算 · RT 由什么决定

开放日 · 缓存与命中率悬崖

提交 · 学生到底在等什么

事故 · 尾延迟

评审 · 代价账本

交付要求

小组完成；截止 10/15（周四）23:59

题面在仓库 2026/quiz/Q-2.md 与 Q-2.html（可打印）

题面已给出独立性假设与提示；今天讲的两页够用

贯穿标准仍是：这项技术解决了什么问题，又增加了什么成本

注 只讲场景钩子和交付要求，不讲题；10/16 讲评

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

四、运行时的层级

第六讲 · 软件运行时

饮水思源 • 爱国荣校

SJTU

重新认识软件 – 软件运行时

运行时的层级

裸机 → 虚拟机 → 容器 → 函数

层	隔离靠什么	启动	密度	解决了什么	增加了什么成本
裸机	物理	分钟 ~ 天	1	性能无损	利用率低、交付慢
虚拟机	hypervisor，各有内核	分钟	十	强隔离、按需开机	每台一份内核与系统开销
容器	namespace + cgroup，共享内核	秒	百	打包即制品、秒级起停、密度高	隔离弱于 VM；共享内核的故障面
函数	平台托管的实例	冷启动 100 ms ~ 秒	按需	缩到零、不管实例	冷启动；时长 / 内存限制；平台锁定

每往下一层，「一个实例」越轻、越短命：L-05 实例轴的时间尺度从月缩到分钟再到毫秒
要求也随之收紧：启动快、无状态、可随时被杀

判断 这项技术解决了什么问题，又增加了什么成本?

来源：Linux namespaces(7)、cgroups(7)；AWS Lambda 文档（execution environment）；数值为量级示意

容器就是被限制的进程

namespace
决定它看得见什么
pid / net / mnt / uts / ipc / user

cgroup
决定它能用多少
cpu / memory / io

镜像
决定它的文件系统
就是 L-05 的不可变制品

没有「容器进程」这种特殊进程：ps 在宿主机上照样看得到它，只是视野和配额被限制了

问题	原因	对策
容器里 CPU 没满，服务却时快时慢	cgroup 的 CPU 配额用完被节流（throttling），每个周期都要停一下	看 throttled 计数；配额按峰值定或去掉硬配额
容器被 OOMKilled，宿主机内存明明够	超的是 cgroup 限制，不是机器	第 7 页的账：堆 + 堆外 + 余量
JVM 在容器里开了 64 个 GC 线程	老版本 JVM 看到的是宿主机的核数	JDK 10+ 自动读 cgroup；或显式指定

要点 容器化没改变「它是一个进程」，只是给 CPU 时间、内存、fd 各装了一个阀门——模块 6 资源管理拧的就是这些阀门

来源：namespaces(7)、cgroups(7)；Kubernetes docs, Resource Management for Pods and Containers

函数（Serverless）与冷启动

平台替你管实例

请求来了才起实例，没请求就缩到零；按调用计费

冷启动 = L-05「启动时间是硬参数」的极端形式

从零到能服务要 100 ms 到几秒，取决于运行时——第二节的表在这里兑现：Go / 原生镜像快，JVM 慢

限制与适用

单次执行时长、内存上限、并发上限

没有本地持久状态——L-05 无状态的强制版

适合：突发、低频、事件驱动（发短信、生成报表、图片处理）

不适合：长连接、需要预热的热路径

L-02 风格比较里 Serverless「省成本、控制有限」——控制有限的具体内容就是这一页

要点 函数把实例轴压缩到一次请求的长度；它能省的和它要求的，都来自这一点

来源：AWS Lambda 文档；L-02 讲义（架构风格比较）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

五、课程实践：单机 Tomcat 的运行时画像

饮水思源 · 爱国荣校

第六讲 · 软件运行时

SJTU

参数设定：按思考题 1 的单机场景

机器与运行时

4 核 8 GB，Tomcat（JVM）一台，MySQL 在另一台（L-04 第一阶段）

Tomcat NIO 连接器默认：maxThreads 200、maxConnections 8192、acceptCount 100

JVM：-Xmx 4 GB；线程栈默认 1 MB 预留

数据库连接池（HikariCP）：默认 10

每个请求与在线人数

一次查询：CPU 3 ms + 等数据库 8 ms + 其余 1 ms ≈ 占线程 12 ms

一次提交：CPU 5 ms + 数据库写 10 ms + 行锁等待（随并发变化）

开放瞬间 2000 人在线，每人 2 ~ 6 条 keep-alive 连接

以上为设定值，与思考题 1 压测表自洽：800 QPS 正常 / 1200 QPS p95 1.8 s / 1500 QPS 崩

钩子 最先崩的是线程池还是行锁？——先把四个上限算出来

四个上限，哪个先到?

资源	参数	每请求占用	上限（读路径）
CPU	4 核	3 ms	4 / 0.003 ≈ 1333 QPS
数据库连接池	10 个	8 ms	10 / 0.008 = 1250 QPS
线程池	200	12 ms	200 / 0.012 ≈ 16 700 QPS
客户端连接 / fd	8192 / ulimit	按在线人数算，不按 QPS	2000 人 × 4 ≈ 8000 条——贴着上限
内存	8 GB	≈ 4 GB 堆 + 0.5 GB 堆外	不按 QPS；看连接数与响应大小

对照思考题 1 压测表：800 QPS 时 CPU 60%、连接池 64%，p95 150 ms 正常；1200 QPS 时两者都到 90%～96%，排队非线性爆炸；1500 QPS 超过上限，崩——数字对得上

要点 读路径最先到顶的是连接池与 CPU，线程池差着一个数量级——「最先崩的是线程池」在读路径上是错觉

课堂讨论

1 读路径

连接池调到 30 之后，下一个到顶的是谁？

再往上还能怎么办（不加机器）？

2 写路径（抢课）

为什么线程池反而会先满？

加到 2000 个线程有用吗？

3 开放瞬间 2000 人 × 4 条连接

碰到的是 maxConnections 还是 ulimit？CPU 此时在干什么？

思考题 1 里的 Nginx 对这件事有什么帮助？

4 换运行时

把服务换成 Go，或者打开虚拟线程：

表里哪些上限变了、哪些没变？

讲法 四题各 2 分钟小组讨论、1 分钟汇报，余下讲评

讨论要点

1 读路径

池 30 → 池上限 3750，下一个是 CPU 1333

不加机器只能降每请求 CPU：缓存热数据（L-04 第二阶段）、少做序列化、查询合并

再往上就是思考题 1 的 4 台 + Nginx

2 写路径

行锁把总吞吐封在约 1200 TPS 以内（单机约 300 TPS），2000 个提交在它前面排队，位置先是线程池后是 accept 队列

2000 个线程只是候车室更大，上下文切换与内存会先把机器拖垮

解在数据层：Redis 原子扣减 / 排队削峰 / 限流

3 连接

ulimit 还是 1024 → 先报 Too many open files，CPU 很闲

提高之后贴着 maxConnections 8192：新连接在 backlog 排队、握手超时

Nginx 替后端持有上万条客户端连接，后端只看到少量复用的上游连接

4 换运行时

线程这一层的上限消失或变得很大

CPU 上限（3 ms/请求）基本不变；连接池上限不变（在数据库那头）；行锁上限不变

换运行时改的是线程成本，不改 CPU、连接池和锁的上限

结论 线程池是症状，行锁是原因；最先到顶的资源要算出来，不是猜出来——下一讲把每种资源的上限讲清楚

本讲小结

- **跑起来的软件是一个进程**
  - 占三样东西：线程（CPU 时间 + 栈）、内存（堆 + 堆外 + 页缓存）、fd（连接 + 文件）
- **选运行时 = 选资源画像**
  - 并发单位、内存基线、启动时间、多核利用各不相同
- **一请求一线程：λ_max = 线程数 / W**
  - 排队无处不在，有界 + 超时才有背压；连接池按数据库能力定
- **线程池是症状，行锁是原因**
  - 非阻塞和虚拟线程只改线程这一层的上限
- **容器 = 被限制的进程；函数 = 启动时间与无状态的极端版**

判断标准 这个东西在机器上占的是什么？上限由哪个资源给？

课后思考与参考资料

课后思考

1. 给你小组项目的服务画一张运行时画像：用速查页的命令量出线程数、RSS、fd 数，按「四个上限」的表估一次哪个上限先到。

2. 思考题 2 第四题要用到的 1 – 0.99^N：首页同步等待的下游调用有几个？算出来。

3. 2025 实验 2 的 Go 后端换成 Java，「四个上限」表里哪些行会变？

参考资料

B. Gregg, Systems Performance: Enterprise and the Cloud, 2nd ed. (2020), Ch. 2、3、6、7

Linux man pages：proc(5)、sched(7)、epoll(7)、namespaces(7)、cgroups(7)、ss(8)

Apache Tomcat 10.1, HTTP Connector（maxThreads / maxConnections / acceptCount）

JEP 444: Virtual Threads；JDK container support (JDK-8146115)

HikariCP wiki, About Pool Sizing；MySQL 8.4 Reference, max_connections

J. Dean & L. Barroso, The Tail at Scale, CACM 56(2), 2013

Kubernetes docs, Resource Management for Pods and Containers；AWS Lambda, Execution environment

The Twelve-Factor App, VIII. Concurrency

感谢聆听

饮 水 思 源　爱 国 荣 校